

Transmission Control Protocol

Module 06

Duration: 27:00

Video: <https://www.youtube.com/watch?v=9bKXrX4lz6g>

Course Coordinator - Dr Maninder Singh, Professor and Dean, Academic Affairs, Thapar University, Patiala

Subject Expert - Dr. Yogesh Chaba, Department of Computer Engineering and IT, Guru Jambheshwar University of Science & Technology, Hisar

Auto-generated YouTube transcript for personal study. Spoken wording belongs to the original lecture. Follow the video link above; this file is not an official CEC/SWAYAM publication.

Hello friends I am Professor yogesh Chaba from Department of computer science and engineering Guru jambheshwar University of Science and Technology Hisar harana I welcome you all to the lecture series of cyber and information security in this lecture we will study about transmission control protocol which is used for connection oriented transmission at transport layer level for end to end connectivity of computers first of all please see the contents which I shall cover after covering brief introduction about TCP I shall take up header of TCP which will be followed by connection management and flow control process in TCP so now let us start the foundation of TCP was laid in May 1974 when Institute of electrical and electronic Engineers itle published a

paper entitled a protocol for packet Network intercommunication the papers author Surf and KH described an internetworking protocol for sharing resources using packet switching among the nodes a central control component of this model was the transmission control program TCP that incorporated both connection oriented links and datagram services between host this transmission control program was later divided into a modular architecture consisting of the transmission control protocol at the connection oriented layer and the Internet Protocol at the internet layer the model became known informally as PCP IP although formerly it was henceforth called the Internet Protocol suit PCP is a reliable stream delivery

service that guarantees that all bytes received will be identical with bytes sent and in the correct order this fundamental technique requires the receiver to respond with an acknowledgement message as it receives the data the sender keeps a record of each packet it sends the sender also maintains a timer from when the packet was sent and retransmits a packet if the timer expires before the message has been acknowledged the timer is needed in case a packet gets lost or corrupted TCP was formally defined in RFC 793 as time went on various errors and inconsistencies were detected and the requirements were changed in some areas these clarifications and some bugs fixes are detailed in RFC 112 extensions are given in RFC 1323 while IP handles actual delivery of

the data TCP keeps track of the individual units of data transmission called segments the internet layer encapsulates each TCP segment into an IP packet by adding an IP header that includes the destination IP address when the client program on the destination computer receives them the TCP layer reassembles the individual visual segments and ensures they are correctly ordered and error free for lack of a better term we will use acronym tpdu that is transport protocol data unit for messages sent from transport layer thus tpd exchanged by the transport layer are contained in packets at the Network layer in turn packets are contained in frames at the data link layer this nesting is Illustrated in

figure TCP service is obtained by the sender and receiver creating end points called sockets each socket has a socket number consisting of the IP address of the host and a 16bit number local to that host called a port a port is the TCP name for a transport service access point tsap a socket may be used for multiple connections at the same time in other words two or more connections May terminate at the same socket connections are identified by the socket identifiers at both ends that is Socket 1 socket 2 no virtual circuit numbers or other identifiers are used port numbers below 1024 are called well-known ports and are reserved for standard Services a few of the better known ones are listed in table the internet

assigned numbers Authority are AA is responsible for maintaining the official assignments of port numbers for specific uses as we can see in table Port 20 supports TCP and UDP communication for FTP data transfer Port 23 supports TCP and UDP communication for tnet protocol Port 25 supports TCP and UDP communication for simple mail transfer protocol SMTP in email system Port 53 supports TCP and UDP communication for domain name system DNS Port 69 is for trial file transfer protocol tftp Port 0 supports TCP sctp and UDP communication for hyper text transfer protocol HTTP Port 107 supports TCP and UDP for remote tet service protocol Port 109 supports TCP and UDP P for post office protocol V2 Port 110 supports TCP and UDP for post office protocol

V3 Port 115 supports TCP for simple file transfer protocol Port 119 supports TCP for Network news transfer protocol nnt P Port 143 supports TCP for internet message access protocol IMAP Port 220 supports TCP and UDP for internet message access protocol IMAP Port 500 supports TCP and UDP for Internet Security Association and Key Management prot protocol Port 530 supports TCP and UDP for remote procedure call RPC Port 554 supports TCP and UDP for realtime streaming protocol rtsp protocol 587 supports TCP for email message submission SMTP Port 698 supports UDP for optimized link state routing osr Port 953 supports TCP and UDP for domain name system R and DC Service Port 995 supports TCP for post

office protocol three connections are full duplex and point to point full duplex means that traffic can go in both directions at the same time point to point means that in each connection has exactly two end points TCP does not support multicasting or broadcasting a TCP connection is a bite stream not a message stream message boundaries are not preserved end

to end for for example if the sending process does four 512 byte right to a TCP stream these data may be delivered to the receiving process as four 512 byte chunks two 1024 by chunks or one 2048 byte chunk as shown in figure or some other way also there is no way for the receiver to detect the the unit in which the data were written figure a shows four 512 by segments sent as separate IP datagram

and figure B shows the 2048 byte of data delivered to the application in a single read call after basic of TCP and Port let us see the TCP segment header figure shows the layout of a TCP segment every segment begins with a fixed format of 20 by header the fixed header may be followed by header options after the options if any up to 65535 -20 -20 which is equal to 65495 databytes may follow where the first 20 refers to the IP header and second 20 refers to the TCP header segments without any data are legal and commonly used for acknowledgements and control messages now let us discuss the TCP header Fields One By One The Source port and destination Port fields identify the local end points of the connection a port plus its host IP

address forms a 48 bit unique end point the source and destination end points together identify the connection the sequence number and acknowledgement number field performs their usual functions for the purpose of delivery confirmation no note that the acknowledgement number specifies The Next Bite expected not the last bite correctly received both are 32 bits long because every bite of data is numbered in a TCP stream the TCP header length tells how many 32bit words are contained in the TCP header this information is needed because the op options field is of variable length technically this field really indicates the start of the data within the segment measured in 32bit words but that number is just the header

length in words so the effect is the same next comes a six-bit field that is not used next are six 1 bit Flags urg is set to one if the Urgent pointer is in use the Urgent pointer is used to indicate a bite offset from the current sequence number at which urgent data are to be found the AK bit is set to one to indicate that the acknowledgement number is valid if AK is zero the segment does not contain an acknowledgement so the acknowledgement number field is ignored the psh bit indicates pushed data the receiver is hereby kindly requested to deliver the data to the application upon arrival and not buffer it until a full buffer has been received the rst bit is used to reset a connection that has become confused due to host crash or some mother

15 minutes, 1 second reason the sin bit is used to establish connections the connection request has sin as one and akk as zero to indicate that the piggy back acknowledgement field is not in use the connection reply does bear an acknowledgement so it has sin as one and AK as one in essence the sin bit is used to denote connection request and connection accepted if AK bit is used to distinguish between those two possibilities the fin bit is used to release a connection it specifies that the sender has no more data to transmit flow control in TC p is handled using a variable sized sliding window the window size field tells how many bytes may be sent starting at the bite acknowledged a check sum is also

provided for extra reliability it checks the header, the data and the conceptual pseudo header after TCP header. Let us move over to socket Primitives in TCP. Socket Primitives are widely used for internet programming; they are listed in Figure. The socket primitive creates a new communication end point. Newly created sockets do not have Network addresses; these are assigned using the bind primitive. The bind primitive is used to attach a local address to a socket. Next comes the listen call, which announces willingness to accept connections and allocates space to queue incoming calls for the case that several clients try to connect at the same time. To block waiting for an incoming connection, the server executes the accept primitive. Now let us look at the client.

Side here to a socket must first be created using the socket primitive, but bind is not required since the address used does not matter to the server. The connect primitive blocks the caller and actively starts the connection process. When the appropriate TPDU is received from the server, the client process is unblocked and the connection is established. Both sides can now use send or receive to transmit and receive data over the full duplex connection. Connection release with socket is symmetric: when both sides have executed a close primitive, the connection is released. Now we shall learn how TCP connection is established. Connections are established in TCP by means of the three-way handshake process to establish a

connection. On one side, say the server, passively waits for an incoming connection by executing the listen and accept primitives. On the other side, say the client, executes a connect PR primitive. The connect primitive sets a TCP segment with a SYN bit on and ACK bit off and waits for a response. When this segment arrives at the destination, the TCP entity checks to see if there is a process that has done a listen on the port given in the destination Port field. If not, it sends a reply with the RST bit on to reject the connection. If some process is listening to the port, that process is given the incoming TCP segment; it can then either accept or reject the connection. If it accepts, an acknowledgement segment is sent.

Back, the sequence of TCP segments sent in the normal case is shown in Figure. A 20-minute, 1-second sequence on the left side shows receipt of SYN with SEQ as X from host to return SYN with SEQ as Y and ACK as $x + 1$. Further, host one sends data with SEQ as $x + 1$ and ACK as $y + 1$. In the event that two hosts simultaneously attempt to establish a connection between the same two sockets, the sequence of events is as illustrated in Part B on the right side of Figure, which clearly shows that collision will occur after connection setup. Let us see how TCP connection is released. To release a connection, either party can send a TCP segment with a FIN bit set, which means that it has no more data to transmit. When the FIN is acknowledged, that direction is shut down.

For new data, data may continue to flow indefinitely in the other direction when both directions have been shut down. The connection is released normally. Four TCP segments are needed to release a connection: one FIN and one ACK for each direction. However, it is possible for the first ACK and the second FIN to be contained in the same segment, reducing the total segment count to three. To avoid the two-army problem, timers are used if a response to a FIN is not

forthcoming within two maximum packet lifetime the sender of the fin releases the connection the other side will eventually notice that nobody seems to be listening to it anymore and will time out as well now let us see TCP connection management and flow

control as mentioned earlier window Management in TCP is not directly tied to acknowledgements as it is in most data link protocols now let us see TCP connection management and flow control process step by step in Step One the receiver has a 4,096 byte buffer as shown in right side of figure if the sender transmits a 2048 byte segment in step one with sequence as zero and it is correctly received the receiver will acknowledge the segment in step two however since receiver has now only 2048 bytes of buffer space it will advertise a window of 2048 starting at the next byte expected and ACK as 2048 at this point 2K space is available in receiver buffer as shown on right side in step two in step three the sender transmits

another 2048 bytes with sequence as 2048 receiver receives this 2K data and its receiver buffer is full so in step four it is acknowledged with ACK as 4096 and win as zero the sender must stop until the application process on the receiving host has removed some data from the buffer at which TCP can advertise a larger window when the window is zero the sender may not normally send segments with two exceptions first Urgent data may be sent for example to allow the user to kill the process running on the remote machine second the sender may send a one byte segment to make the receiver will re-announce the next byte expected and window size the TCP standard explicitly provides this option to prevent deadlock if a window announcement ever gets

lost as soon as receiver application reads 2K data it has 2K window size so receiver replies with ACK as 4096 and win as 2048 in Next Step transmitter will send 1K data with sequence as 4096 so in this way communication takes place and end-to-end flow control through TCP so friends this was transmission control protocol finally summarizing we learned about introduction to TCP protocol which was followed by TCP header in the end we discussed connection management and flow control in TCP hope the concepts explained in this lecture were understood and helpful hope to see you in the next lecture till then goodbye enjoy the day thank you